



LiveVal: Real-time and Trajectory-based Data Valuation via Adaptive Reference Points

Jie Xu

jiexu49-c@my.cityu.edu.hk
City University of Hong Kong
Department of Computer Science
Kowloon, Hong Kong

Cong Wang

congwang@cityu.edu.hk
City University of Hong Kong
Department of Computer Science
Kowloon, Hong Kong

Zihan Wu

zihanwu7-c@my.cityu.edu.hk
City University of Hong Kong
Department of Electrical Engineering
Kowloon, Hong Kong

Xiaohua Jia

csjia@cityu.edu.hk
City University of Hong Kong
Department of Computer Science
Kowloon, Hong Kong

Abstract

Data valuation quantifies the contribution of each training data, enabling harmful data detection and enhancing model robustness. However, existing methods are typically post-hoc and require fully trained models, making them computationally expensive and unable to detect harmful data early in training. We propose LiveVal, a real-time and trajectory-based data valuation method that assesses training data by analyzing their influence on the optimization trajectory. LiveVal includes three key innovations: 1) a real-time valuation framework with minimal overhead, seamlessly integrated into standard training processes; 2) an adaptive reference point mechanism that assesses data impact on generalization; and 3) a normalization technique that ensures fair comparisons across training stages. Theoretical analysis shows that LiveVal achieves directional alignment, boundedness, stability, and fairness. Experiments demonstrate that LiveVal achieves up to $180\times$ speedup over baseline methods while maintaining robust performance across diverse models and datasets.

CCS Concepts

• Information systems → Data cleaning; • Computing methodologies → Online learning settings.

Keywords

Real-time Data Valuation, Data Influence Analysis, Data Cleansing

ACM Reference Format:

Jie Xu, Zihan Wu, Cong Wang, and Xiaohua Jia. 2025. LiveVal: Real-time and Trajectory-based Data Valuation via Adaptive Reference Points. In *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM '25)*, November 10–14, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3746252.3761138>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

CIKM '25, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 979-8-4007-2040-6/2025/11
<https://doi.org/10.1145/3746252.3761138>

1 Introduction

Training data points contribute differently to model learning. Some enhance generalization, while others introduce harmful noise that degrades performance. Data valuation, which quantifies the contribution of each data point, is critical for robust and efficient machine learning [38, 39].

Traditional data valuation methods, including Leave-One-Out (LOO), Shapley Value [10, 17, 21], influence functions [4, 18], measure data value by how test performance changes when a data point is removed. While intuitive, these post-hoc approaches have fundamental limitations. 1) These methods evaluate data influence only on the final trained models, preventing early detection of harmful training data. When harmful data can be identified, it may have already corrupted model optimization irreversibly [19, 30]. 2) They often misjudge high-loss data as low-value data. Challenging yet informative data are high-loss but high-value, since they enhance generalization by enabling robust representations and precise decision boundaries [6, 12, 49]. 3) They are computationally intensive because they require model retraining [10], second-order approximations [18, 20], or full gradient trajectory storage [33], to evaluate each data point's influence. This high cost makes them impractical for real-time or dynamic training scenarios.

To overcome these limitations, we propose LiveVal, a real-time and trajectory-based valuation approach. Unlike existing methods, LiveVal continuously assesses how each data point guides the optimization trajectory toward better generalization, thereby enabling early detection of harmful data without requiring convergence.

Despite its promise, trajectory-based data valuation poses significant challenges. First, it is computationally expensive to track each data point's influence over the entire training trajectory. This requires storing or recomputing training data gradients at every training step, resulting in prohibitive overhead for large datasets or long training periods. Second, no established criteria exist to reliably determine whether a training data point contributes positively to the optimization process toward better generalization. Third, data used in the early training stage may receive unfairly high values. Since gradient magnitudes naturally decrease during training, data used earlier will produce larger parameter updates and receive higher values regardless of their intrinsic quality.

To address these challenges, LiveVal develops three key innovations. First, LiveVal seamlessly integrates with standard optimization processes, eliminating expensive post-hoc recomputation. Second, LiveVal employs an adaptive reference point mechanism that provides reliable benchmarks for evaluating data contributions to optimization progress. Third, LiveVal normalizes contributions across training stages to compensate for gradient decay, ensuring fair valuation throughout learning.

The main contributions of this paper can be summarized as:

- We propose LiveVal, a real-time trajectory-based data valuation framework that evaluates data contributions during training without requiring model convergence.
- We design an adaptive reference point mechanism with normalization that enables fair and reliable data valuation across different training stages, addressing gradient decay bias.
- We provide theoretical guarantees showing LiveVal produces directionally aligned, bounded, stable, and fair valuations. Extensive experiments show that LiveVal achieves up to 180× speedup over baseline methods while maintaining robust performance across diverse models and datasets.

2 Related Work

LOO methods [5] evaluate data value by retraining models without individual data points but suffer from prohibitive computational costs and overlook inter-data dependencies [37]. Shapley Value-based methods [36, 45] address this by evaluating marginal contributions across all data subsets, but their exponential computational costs make them impractical for large-scale applications [10, 21, 40]. Multiple approximation approaches attempt to make Shapley-based methods more practical. Data Shapley [10] employs Monte Carlo sampling, while G-Shapley [10] leverages gradient contributions; however, both require model convergence and multiple retrains. OFA [26] improves convergence rates through an optimized sample vector. Other techniques [9, 11, 16, 28, 50] still require extensive computations and remain constrained by convergence dependencies. Recent work [27, 37, 42] further reveals that these LOO and Shapley value-based methods are sensitive to the randomness of retraining and produce inconsistent data value rankings across different runs. Some approaches aim to avoid multiple subset evaluations and retraining. Data-OOB [22] eliminates retraining by reusing out-of-bag predictions, but it is limited to bagging-based models and depends on ensemble quality. In-Run Data Shapley [43] estimates data contributions during a single training run through Taylor approximations but inherits order sensitivity from gradient-based methods and cannot guarantee fairness in reflecting data contributions to global model performance across training stages. The Shapley concept has also been applied in other contexts, with Active Data Shapley [11] for efficient batch active learning, while in feature attribution domains, WeightedSHAP [23] generalizes the Shapley approach for explaining feature importance rather than data valuation.

Influence functions [18] approximate LOO but rely on assumptions rarely satisfied in modern deep learning, such as loss convexity and model optimality [2, 14]. Although recent extensions [3, 13, 20, 35] propose approximations for non-convex scenarios, challenges

remain for real-time analysis in deep architectures [2]. Recent findings [1] show that influence functions capture only local model changes and fail to approximate LOO, leading to inaccurate evaluation of data influence.

Gradient-based methods such as TracIn [33], GraND [32], and MOSO [41] evaluate data by analyzing gradient contributions during training, avoiding expensive retraining. However, they suffer from training order bias, where earlier data receive unfairly high valuations due to larger initial gradient magnitudes. CTD [44] further quantifies how training data position in the optimization sequence affects its influence, formalizing trajectory-specific leave-one-out influence estimation. CGSV [47] and other cosine similarity methods [8, 29, 46] focus on local gradient alignments at individual iterations but lack mechanisms to assess long-term effects on model convergence. In addition, there are also approaches to analyze data influence through proxy models [15, 48], although this introduces the overhead of auxiliary model training.

3 Preliminaries and Problem Definition

3.1 Preliminaries

We consider the standard supervised learning setting with a training dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ consisting of N data points. Here, $\mathbf{x}_i \in \mathcal{X}$ denotes the input features, and $y_i \in \mathcal{Y}$ denotes the corresponding labels. Let $f(\mathbf{x}; \theta)$ be a model parameterized by $\theta \in \mathbb{R}^d$, and let $\ell(\cdot, \cdot)$ denote the loss function. The learning objective is to find the parameters that minimize the empirical risk:

$$\theta^* = \arg \min_{\theta \in \Theta} \frac{1}{N} \sum_{i=1}^N \ell(f(\mathbf{x}_i; \theta), y_i). \quad (1)$$

Stochastic Gradient Descent (SGD) [34] is a widely used optimization algorithm that solves the empirical risk minimization problem by iteratively updating the model parameters using mini-batch gradients. For consistency with prior data valuation studies [10, 18, 33], we also adopt SGD in this work. SGD updates the model parameters at iteration t as:

$$\theta_t = \theta_{t-1} - \eta_t \frac{1}{\|\mathcal{B}_t\|} \sum_{i \in \mathcal{B}_t} \nabla_{\theta} \ell(f(\mathbf{x}_i; \theta_{t-1}), y_i), \quad (2)$$

where η_t is the learning rate, $\mathcal{B}_t \subset \mathcal{D}$ denotes the mini-batch randomly selected at iteration t , and $\nabla_{\theta} \ell(f(\mathbf{x}_i; \theta_{t-1}), y_i)$ is the gradient of the loss.

This iterative update generates a sequence of parameters $\{\theta_t\}_{t=0}^T$, known as the *parameter trajectory*, where θ_0 is the initialized model parameters and θ_T is the final model parameters. This trajectory captures the evolution of the model parameters throughout the entire training process.

3.2 Problem Definition

We aim to evaluate the contribution of individual training data to the model's learning process in real-time based on their influence on the optimization trajectory. Specifically, the method should evaluate whether a data point's gradient update moves the model toward regions in the parameter space associated with better generalization, which can be approximated using a reference point θ_{ref}^t at each step t . Training data are considered more valuable if their gradient updates move the model parameters closer to the reference point.

Formally, for a training data $(\mathbf{x}_i, y_i)$ included in the mini-batch $\mathcal{B}_t$ at step t , its real-time value can be defined as:

$$v_i^t = \mathcal{V}_t((\mathbf{x}_i, y_i), \theta_{t-1}, \theta_{\text{ref}}^t), \quad (3)$$

where θ_{ref}^t is a reference point that serves as a proxy for improved generalization, and $\mathcal{V}_t(\cdot)$ quantifies how effectively the data guides the model toward this improved state in the parameter space.

The cumulative value of a data point across the entire training process is then defined as: $v_i^{[0,T]} = \sum_{t=1}^T v_i^t$, which captures its total influence on model training from initialization to completion.

The method must satisfy three key requirements: 1) computational and storage efficiency that enables real-time data valuation during training without requiring model convergence or retraining, 2) cross-stage fairness that ensures consistent value comparisons across different training phases despite natural variations in gradient magnitudes and parameter sensitivity, and 3) trajectory alignment that captures each data point’s contribution to the optimization trajectory.

4 Trajectory-Based Data Valuation Framework

In this section, we propose a trajectory-based data valuation framework, which quantifies the influence of training data based on their guidance along the model’s optimization trajectory. The section begins with motivation and design principles, then describes a basic method using static reference points, and finally extends to LiveVal using adaptive reference points for real-time data valuation.

4.1 Motivation and Design Principles

Most data valuation methods [10, 17, 18] evaluate the contribution of training data by comparing model performance (e.g., test loss) with and without specific data points. These loss-based approaches suffer from several limitations, including inconsistent valuations [42], sensitivity to the choice of the loss function, high re-training costs [10], and dependence on full model convergence [18]. Most importantly, these methods provide only a static snapshot of data utility, failing to capture how data dynamically influences the optimization process. This static perspective systematically undervalues difficult-but-informative training data (e.g., rare-class or decision-boundary points) that initially cause high prediction errors but become invaluable once the model develops sufficient representational capacity [12, 49].

While recent gradient-based methods such as TracIn [33] and GraND [32] incorporate training dynamics into their valuation schemes, they operate with a local perspective. These approaches evaluate gradient similarity or magnitude at specific model states, capturing only instantaneous gradient contributions without establishing connections to long-term influence. Consequently, they fail to assess whether immediate gradient contributions align with the overall optimization objective or merely reflect training fluctuations.

To address these limitations, we propose a trajectory-based data valuation framework that measures data value through its dynamic influence on the entire optimization trajectory. In this framework, training data are considered valuable if they consistently guide the model parameters toward regions of the parameter space associated with better generalization. In practice, this desirable region

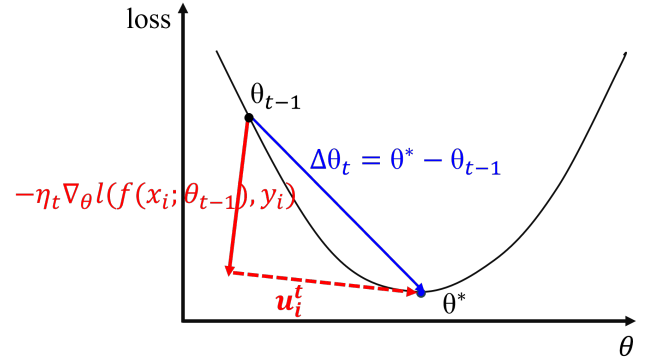


Figure 1: Basic method using the final model parameter as static reference points.

can be approximated by the final converged parameter θ^* , under the assumption that the optimized model reflects improved generalization capabilities. Then, data contributions can be evaluated based on how effectively each training data point moves the model toward the optimal parameters.

4.2 Basic Method with Static Reference Points

The optimization process constitutes a trajectory in parameter space, where training data contribute through gradient computations at each iteration. Different data points have different influences on parameter update direction and magnitude, with their cumulative effects shaping the overall trajectory. During SGD, key information at each iteration t , including θ_{t-1} , η_t , and $\mathcal{B}_t$ is stored.

To evaluate each training data point’s contribution to this trajectory, we measure how effectively it guides the optimization toward the final converged parameter. For data point $i \in \mathcal{B}_t$, we first establish the optimal direction toward the final model parameter:

$$\Delta \theta_t = \theta^* - \theta_{t-1}, \quad (4)$$

which represents the ideal optimization direction from the current state.

To assess each data point’s effectiveness in moving toward this target, we introduce a hypothetical state $\mathbf{u}_i^t$ that would result from applying only that data point’s gradient update.

$$\mathbf{u}_i^t = \theta^* - (\theta_{t-1} - \eta_t \nabla_{\theta} \ell(f(\mathbf{x}_i; \theta_{t-1}), y_i)). \quad (5)$$

As shown in Figure 1, $\|\mathbf{u}_i^t\|$ captures the remaining distance to θ^* after this hypothetical update. A reduction in the distance ($\|\mathbf{u}_i^t\| < \|\Delta \theta_t\|$) indicates the data point moves the model closer to the target, while an increase suggests it moves away from the desired direction.

However, direct distance comparisons across different training stages are unfair due to naturally varying gradient magnitudes. To address this, the step-wise value is defined with normalization:

$$v_i^t = \frac{\|\Delta \theta_t\| - \|\mathbf{u}_i^t\|}{\|\Delta \theta_t\| + \|\mathbf{u}_i^t\|}. \quad (6)$$

The numerator $\|\Delta \theta_t\| - \|\mathbf{u}_i^t\|$ measures improvement toward the reference point. A positive value indicates the data helps guide

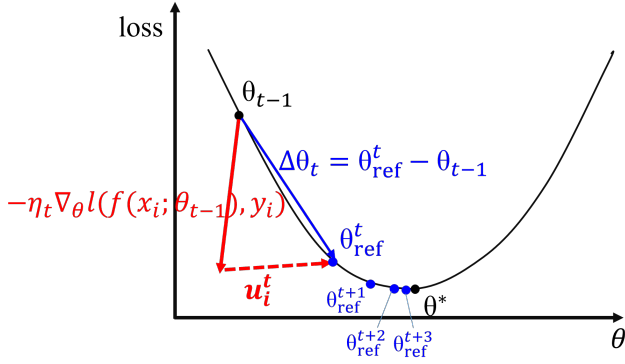


Figure 2: LiveVal's adaptive reference point mechanism.

the model toward the reference point, while a negative value suggests it pushes the model away. The denominator $\|\Delta\theta_t\| + \|u_i^t\|$ provides scale normalization, bounding values within $[-1, 1]$ to enable fair comparison across training stages. The detailed analysis is presented in Section 5.

The cumulative value $v_i^{[0,T]} = \sum_{t=1}^T v_i^t$, where $v_i^t = 0$ if $i \notin \mathcal{B}_t$, captures each data point's total contribution throughout training.

4.3 LiveVal: Real-Time Valuation with Adaptive Reference Points

While the basic method provides valuable insights, it relies on the final model parameters for valuation, making it a post-hoc analysis and unsuitable for real-time analysis. Moreover, it requires storing the entire optimization trajectory for post-hoc analysis, which incurs significant storage overhead.

To address these limitations, we propose LiveVal, a real-time valuation method that employs an adaptive reference point mechanism. Real-time means the ability to provide data influence analysis during training, enabling early harmful data detection without waiting for full convergence. Instead of using the final parameter θ^* as reference points, LiveVal adaptively selects near-future model parameters as reference points, thus enabling real-time analysis during training.

The adaptive reference point mechanism comprises two core components. 1) Reference point selection determines which specific future parameter $\theta_{t-1+\delta_t}$ to use at each step. 2) Dynamic window adjustment optimizes the window size δ_t , expanding during steep gradients and shrinking near convergence, as shown in Figure 2.

4.3.1 Reference Point Selection. At each step t , the reference point θ_{ref}^t is a future model parameter:

$$\theta_{\text{ref}}^t = \begin{cases} \theta_{t-1+\delta_t} & \text{if } t-1+\delta_t < T, \\ \theta_T & \text{otherwise,} \end{cases} \quad (7)$$

where δ_t is the look-ahead window size, and T is the total number of training steps.

This dynamic selection avoids waiting for full convergence while maintaining storage efficiency by only storing parameters within the window. The key challenge lies in adapting the optimal window

size δ_t at each training step to balance evaluation accuracy with computational efficiency.

4.3.2 Dynamic Window Adjustment. A fixed window size cannot adapt to the varying dynamics of model training. Early training requires longer windows to smooth noisy updates, while near convergence, shorter windows capture subtle progress more effectively. LiveVal adjusts the look-ahead window δ_t at each step t based on the loss change rate $\dot{L}$:

$$\dot{L} = \frac{L_{t'} - L_{t-1}}{\delta_{t-1}}, \quad (8)$$

where L_{t-1} is the average training loss at step $t-1$, and $L_{t'}$ is the loss at a future step:

$$L_{t-1} = \frac{1}{\|\mathcal{B}_{t-1}\|} \sum_{i \in \mathcal{B}_{t-1}} \ell(f(\mathbf{x}_i; \theta_{t-1}), y_i), \quad (9)$$

$$L_{t'} = \frac{1}{\|\mathcal{B}_{t'}\|} \sum_{i \in \mathcal{B}_{t'}} \ell(f(\mathbf{x}_i; \theta_{t'}), y_i). \quad (10)$$

Here, $t' = \min(t-1+\delta_{t-1}, T)$ denotes the future step used as the reference point for computing the rate of loss change, bounded by the total training steps T .

Based on the magnitude of $\dot{L}$, the window size is updated as:

$$\delta_t = \begin{cases} \min(\delta_{t-1} + \Delta\delta, \delta_{\max}) & \text{if } \|\dot{L}\| > \epsilon_{\max}, \\ \max(\delta_{t-1} - \Delta\delta, \delta_{\min}) & \text{if } \|\dot{L}\| < \epsilon_{\min}, \\ \delta_{t-1} & \text{otherwise,} \end{cases} \quad (11)$$

where $\Delta\delta$ controls the adaptation rate, $\delta_{\min}$ and $\delta_{\max}$ bound the look-ahead window size, and thresholds $\epsilon_{\min}$, $\epsilon_{\max}$ trigger adjustments.

When the loss changes rapidly ($\|\dot{L}\| > \epsilon_{\max}$), indicating volatile training dynamics, the window size is increased to obtain more stable reference points and smooth out short-term fluctuations. Conversely, when the loss stabilizes near convergence ($\|\dot{L}\| < \epsilon_{\min}$), the window shrinks to capture fine-grained progress with higher temporal resolution. When the rate of loss change falls between these two thresholds, the window size remains unchanged, maintaining a balance between sensitivity to immediate fluctuations and awareness of long-term progress.

With the adaptively selected reference point θ_{ref}^t , LiveVal computes the data value as follows:

$$\Delta\theta_t = \theta_{\text{ref}}^t - \theta_{t-1}, \quad (12)$$

$$u_i^t = \theta_{\text{ref}}^t - (\theta_{t-1} - \eta_t \nabla_{\theta} \ell(f(\mathbf{x}_i; \theta_{t-1}), y_i)), \quad (13)$$

$$v_i^t = \frac{\|\Delta\theta_t\| - \|u_i^t\|}{\|\Delta\theta_t\| + \|u_i^t\|}, \quad v_i^{[0,T]} = \sum_{t=1}^T v_i^t. \quad (14)$$

4.4 LiveVal Implementation via Dual Queues

LiveVal is implemented using a dual-queue architecture (Algorithm 1) that enables real-time valuation during training without storing complete optimization trajectories.

The system maintains two synchronized queues: a Model Queue Q_{θ} storing recent training states $(t, \theta_t, \mathcal{B}_t, \eta_t, L_t)$ within a sliding window $[t - \delta_{\max}, t]$, and a Reference Queue Q_{ref} tracking pending evaluations as $(t_{\text{eval}}, t_{\text{ref}})$ pairs, where t_{eval} is the step to evaluate and t_{ref} is its reference step. During each training step t , LiveVal

Algorithm 1 LiveVal: Real-time Data Valuation

Require: Training dataset $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$, Total steps T , Initial window δ_0 , Window bounds $\delta_{\min}$, $\delta_{\max}$, Adaptation rate $\Delta\delta$, Loss thresholds $\varepsilon_{\min}$, $\varepsilon_{\max}$

Ensure: Step-wise values $\{v_i^t\}_{t=1}^T$, Cumulative value $v_i^{[0,T]}$

```

1: Initialize  $v_i^{[0,T]} \leftarrow 0$  for all  $i$ 
2: Initialize array  $\{v_i^t\}_{t=1}^T \leftarrow 0$  for all  $i$ 
3: Initialize  $\theta_0$  and  $L_0$ 
4: Initialize window size array  $\{\delta_t\}_{t=1}^T \leftarrow \{\delta_0\}$ 
5: Initialize model queue  $Q_\theta$  for storing model parameters
6: Initialize reference queue  $Q_{\text{ref}}$  for storing  $(t_{\text{eval}}, t_{\text{ref}})$  pairs
7: for  $t = 1$  to  $T$  do
8:   Sample mini-batch  $\mathcal{B}_t$ 
9:    $\mathbf{g}_t \leftarrow \frac{1}{|\mathcal{B}_t|} \sum_{i \in \mathcal{B}_t} \nabla_{\theta} \ell(f(\mathbf{x}_i; \theta_{t-1}), y_i)$ 
10:   $\theta_t \leftarrow \theta_{t-1} - \eta_t \mathbf{g}_t$ 
11:   $L_t \leftarrow \frac{1}{|\mathcal{B}_t|} \sum_{i \in \mathcal{B}_t} \ell(f(\mathbf{x}_i; \theta_t), y_i)$ 
12:  Store  $(t, \theta_t, \mathcal{B}_t, \eta_t, L_t)$  in  $Q_\theta$ 
13:   $t_{\text{ref}} \leftarrow \min(t - 1 + \delta_{t-1}, T)$ 
14:  Add  $(t, t_{\text{ref}})$  to  $Q_{\text{ref}}$ 
15:   $\dot{L} \leftarrow (L_t - L_{t-1}) / \delta_{t-1}$ 
16:  if  $\|\dot{L}\| > \varepsilon_{\max}$  then
17:     $\delta_t \leftarrow \min(\delta_{t-1} + \Delta\delta, \delta_{\max})$ 
18:  else if  $\|\dot{L}\| < \varepsilon_{\min}$  then
19:     $\delta_t \leftarrow \max(\delta_{t-1} - \Delta\delta, \delta_{\min})$ 
20:  else
21:     $\delta_t \leftarrow \delta_{t-1}$ 
22:  end if
23:  if  $\exists (t', t) \in Q_{\text{ref}}$  then
24:    Get all pairs  $(t', t)$  from  $Q_{\text{ref}}$ 
25:    for each  $(t', t)$  pair do
26:      Get  $\theta_{t'-1}$  from  $Q_\theta$ 
27:       $\Delta\theta_{t'} \leftarrow \theta_t - \theta_{t'-1}$ 
28:      for each  $i \in \mathcal{B}_{t'}$  do
29:         $\mathbf{g}_i \leftarrow \nabla_{\theta} \ell(f(\mathbf{x}_i; \theta_{t'-1}), y_i)$ 
30:         $\mathbf{u}_i^{t'} \leftarrow \theta_t - (\theta_{t'-1} - \eta_{t'} \mathbf{g}_i)$ 
31:         $v_i^{t'} \leftarrow \frac{\|\Delta\theta_{t'}\| - \|\mathbf{u}_i^{t'}\|}{\|\Delta\theta_{t'}\| + \|\mathbf{u}_i^{t'}\|}$ 
32:         $v_i^{[0,T]} \leftarrow v_i^{[0,T]} + v_i^{t'}$ 
33:      end for
34:      Remove  $(t', t)$  from  $Q_{\text{ref}}$ 
35:    end for
36:  end if
37:  Remove from  $Q_\theta$  where step  $< t - \delta_{\max}$ 
38:  Remove from  $Q_{\text{ref}}$  where  $t_{\text{ref}} < t$ 
39: end for
40: return  $\{v_i^t\}_{t=1}^T, v_i^{[0,T]}$ 

```

checks whether the current parameters θ_t serve as reference points for any pending evaluations. When matches are found, it computes data values for all training data in the corresponding historical batches. Processed states are then removed from the queues to maintain storage efficiency.

This design reduces storage from $O(Td)$ full trajectory storage to $O(\delta_{\max}d)$, where d is the parameter dimension and $\delta_{\max}$ is the

maximum adaptive window size. The computational complexity is $O(TBd)$ over T training steps with batch size B . Unlike existing methods requiring model retraining or post-hoc analysis, LiveVal provides real-time analysis, enabling early detection of harmful data during training.

5 Theoretical Analysis

We establish the theoretical foundations of LiveVal through three key properties: reliability, fairness, and stability. Our analysis relies on standard assumptions in optimization theory.

ASSUMPTION 1 (SMOOTHNESS). *The loss function $\ell(f(\mathbf{x}; \theta), y)$ is β -smooth: $\|\nabla_{\theta} \ell(f(\mathbf{x}; \theta), y) - \nabla_{\theta} \ell(f(\mathbf{x}; \theta'), y)\| \leq \beta \|\theta - \theta'\|$.*

ASSUMPTION 2 (GRADIENT BOUND). *Stochastic gradients are clipped and bounded $G > 0$ ¹, i.e. $\|\nabla_{\theta} \ell(f(\mathbf{x}; \theta), y)\| \leq G$.*

ASSUMPTION 3 (STABLE STEP SIZE). *For any t , the learning rate η_t satisfies $\eta_t G \leq \|\Delta\theta_t\|$.*

5.1 Fundamental Properties of LiveVal

THEOREM 5.1 (FUNDAMENTAL PROPERTIES OF LIVEVAL). *For any training data i and iteration t , the step-wise data value v_i^t satisfies:*

- (1) **Directional Alignment:** *If the gradient update moves parameters closer to the reference point $\theta_{t_{\text{ref}}}^t$ then $v_i^t \geq 0$.*
- (2) **Value Boundedness:** *For any training data i and iterations t , $v_i^t \in [-1, 1]$.*

PROOF. 1) *Directional Alignment:* If the gradient update moves parameters closer to $\theta_{t_{\text{ref}}}^t$, then:

$$\|\theta_{t_{\text{ref}}}^t - (\theta_{t-1} - \eta_t \nabla_{\theta} \ell(f(\mathbf{x}_i; \theta_{t-1}), y_i))\| \leq \|\theta_{t_{\text{ref}}}^t - \theta_{t-1}\|. \quad (15)$$

Equation (15) is equivalent to $\|\mathbf{u}_i^t\| \leq \|\Delta\theta_t\|$, which implies $\|\Delta\theta_t\| - \|\mathbf{u}_i^t\| \geq 0$. Therefore, $v_i^t \geq 0$ as the denominator is positive.

2) *Value Boundedness:* Since $\|\mathbf{u}_i^t\| \geq 0$, we have:

$$\|\Delta\theta_t\| - \|\mathbf{u}_i^t\| \leq \|\Delta\theta_t\| + \|\mathbf{u}_i^t\| \Rightarrow v_i^t \leq 1 \quad (16)$$

$$\|\mathbf{u}_i^t\| - \|\Delta\theta_t\| \leq \|\Delta\theta_t\| + \|\mathbf{u}_i^t\| \Rightarrow v_i^t \geq -1 \quad (17)$$

Therefore, $v_i^t \in [-1, 1]$. $\square$

These properties ensure that LiveVal assigns positive values to beneficial updates and negative values to harmful ones while maintaining consistent scaling across different training stages.

5.2 Stability Analysis of LiveVal

We analyze how stochasticity from mini-batch sampling and random initialization affects LiveVal.

Definition 5.2 (Data Valuation Volatility). *The volatility of data valuation v_i^t for training data i at step t is:*

$$\sigma_i^{[t]} = \sqrt{\mathbb{E} \left[\left(v_i^t - \mathbb{E}[v_i^t] \right)^2 \right]}, \quad (18)$$

where the expectation is overall training randomness.

¹Gradient clipping is a standard practice in deep networks to prevent exploding gradients [31].

THEOREM 5.3 (STABILITY UNDER STOCHASTIC TRAINING). *Under Assumptions 1–3 and $\|\Delta\theta_t\| \geq D_{\min} > 0$, the volatility of v_i^t is bounded by*

$$\sigma_i^{[t]} \leq \frac{\eta_t G}{\|\Delta\theta_t\|} \leq \frac{\eta_t G}{D_{\min}}. \quad (19)$$

PROOF. *Step 1.* Bound the change of a gradient update on parameter distance. Given $\mathbf{u}_i^t = \Delta\theta_t + \eta_t \nabla_{\theta} \ell(f(\mathbf{x}_i; \theta_{t-1}), y_i)$, using the reverse triangle inequality and the bounded gradient assumption, we have:

$$\| \|\mathbf{u}_i^t\| - \|\Delta\theta_t\| \| \leq \| \mathbf{u}_i^t - \Delta\theta_t \| = \eta_t \| \nabla_{\theta} \ell(f(\mathbf{x}_i; \theta_{t-1}), y_i) \| \leq \eta_t G. \quad (20)$$

Step 2. Bound the denominator $v_i^t = \frac{\|\Delta\theta_t\| - \|\mathbf{u}_i^t\|}{\|\Delta\theta_t\| + \|\mathbf{u}_i^t\|}$. From the triangle inequality:

$$\|\mathbf{u}_i^t\| = \|\Delta\theta_t + \eta_t \nabla_{\theta} \ell(f(\mathbf{x}_i; \theta_{t-1}), y_i)\| \quad (21)$$

$$\geq \|\Delta\theta_t\| - \eta_t \|\nabla_{\theta} \ell(f(\mathbf{x}_i; \theta_{t-1}), y_i)\| \quad (22)$$

$$\geq \|\Delta\theta_t\| - \eta_t G \quad (23)$$

Therefore, $\|\Delta\theta_t\| + \|\mathbf{u}_i^t\| \geq 2\|\Delta\theta_t\| - \eta_t G$.

Under the assumption $\eta_t G \leq \|\Delta\theta_t\|$, we obtain $\|\Delta\theta_t\| + \|\mathbf{u}_i^t\| \geq \|\Delta\theta_t\|$. Combining with Step 1:

$$\|v_i^t\| = \frac{\|\|\Delta\theta_t\| - \|\mathbf{u}_i^t\|\|}{\|\Delta\theta_t\| + \|\mathbf{u}_i^t\|} \leq \frac{\eta_t G}{\|\Delta\theta_t\|}. \quad (24)$$

Step 3. Bound the volatility using variance decomposition.

$$\text{Var}(v_i^t) = \mathbb{E} \left[(v_i^t - \mathbb{E}[v_i^t])^2 \right] = \mathbb{E} \left[(v_i^t)^2 \right] - (\mathbb{E}[v_i^t])^2 \leq \mathbb{E} \left[(v_i^t)^2 \right]. \quad (25)$$

Using the bound from Step 2:

$$\sigma_i^{[t]} = \sqrt{\text{Var}(v_i^t)} \leq \sqrt{\mathbb{E} \left[(v_i^t)^2 \right]} \leq \frac{\eta_t G}{\|\Delta\theta_t\|}. \quad (26)$$

□

This bound shows that volatility decreases proportionally with the learning rate η_t , ensuring increasingly reliable valuations as training progresses with learning rate decay.

5.3 Fairness Analysis of LiveVal

Temporal fairness ensures consistent valuations regardless of when data appears. Traditional gradient-based methods often bias early data with larger gradients, undervaluing late-stage contributions. We show LiveVal achieves temporal fairness by making valuation independent of data position in training.

Definition 5.4 (Temporal Fairness). A data valuation method satisfies temporal fairness if training data with equivalent optimization contributions receive equal valuations regardless of their temporal position, given identical gradients and relative orientations to reference vectors.

THEOREM 5.5 (TEMPORAL FAIRNESS OF LIVEVAL). *Let training data i and j be used at steps t_1 and t_2 , respectively. If:*

- (1) $\nabla_{\theta} \ell(f(\mathbf{x}_i; \theta_{t_1-1}), y_i) = \nabla_{\theta} \ell(f(\mathbf{x}_j; \theta_{t_2-1}), y_j) = \mathbf{g}$,
- (2) $\eta_{t_1} = \eta_{t_2} = \eta$,
- (3) $\|\theta_{ref}^{t_1} - \theta_{t_1-1}\| = \|\theta_{ref}^{t_2} - \theta_{t_2-1}\| = D$,
- (4) $\frac{\langle \Delta\theta_{t_1}, \mathbf{g} \rangle}{\|\Delta\theta_{t_1}\| \|\mathbf{g}\|} = \frac{\langle \Delta\theta_{t_2}, \mathbf{g} \rangle}{\|\Delta\theta_{t_2}\| \|\mathbf{g}\|}$,

then LiveVal assigns equal valuations: $v_i^{t_1} = v_j^{t_2}$.

PROOF. Given $\mathbf{u}_i^t = \Delta\theta_t + \eta_t \nabla_{\theta} \ell(f(\mathbf{x}_i; \theta_{t-1}), y_i)$ and

$$v_i^t = \frac{\|\Delta\theta_t\| - \|\mathbf{u}_i^t\|}{\|\Delta\theta_t\| + \|\mathbf{u}_i^t\|}. \quad (27)$$

Under the given conditions:

$$\mathbf{u}_i^{t_1} = \Delta\theta_{t_1} + \eta \mathbf{g}, \quad \mathbf{u}_j^{t_2} = \Delta\theta_{t_2} + \eta \mathbf{g}. \quad (28)$$

Using the law of cosines:

$$\|\mathbf{u}_i^{t_1}\|^2 = D^2 + \eta^2 \|\mathbf{g}\|^2 + 2\eta \langle \Delta\theta_{t_1}, \mathbf{g} \rangle, \quad (29)$$

$$\|\mathbf{u}_j^{t_2}\|^2 = D^2 + \eta^2 \|\mathbf{g}\|^2 + 2\eta \langle \Delta\theta_{t_2}, \mathbf{g} \rangle. \quad (30)$$

From condition 4 and the fact that $\|\Delta\theta_{t_1}\| = \|\Delta\theta_{t_2}\| = D$, we have $\langle \Delta\theta_{t_1}, \mathbf{g} \rangle = \langle \Delta\theta_{t_2}, \mathbf{g} \rangle$. Thus, $\|\mathbf{u}_i^{t_1}\| = \|\mathbf{u}_j^{t_2}\|$, which implies:

$$v_i^{t_1} = \frac{D - \|\mathbf{u}_i^{t_1}\|}{D + \|\mathbf{u}_i^{t_1}\|} = \frac{D - \|\mathbf{u}_j^{t_2}\|}{D + \|\mathbf{u}_j^{t_2}\|} = v_j^{t_2}. \quad (31)$$

□

This property ensures that LiveVal evaluates data based on their actual optimization contributions rather than temporal position, addressing systematic bias in gradient-based methods.

6 Experiments

We evaluate LiveVal across three dimensions: accuracy and efficiency in identifying corrupted data (Sections 6.2.2 and 6.3), robustness across data qualities and modalities (Sections 6.4 and 6.6), and scalability to large models (Section 6.7).

6.1 Experimental Setup

6.1.1 Datasets and Models. We evaluate LiveVal across three data modalities: tabular (Adult [7]), text (20 Newsgroups [24]), and image (MNIST [25]) data. We test on three models of increasing complexity: Deep Neural Network (DNN), Convolutional Neural Network (CNN, LeNet-5), and Inception V3 (24M parameters). Full implementation details are available at <https://github.com/liveval/liveval>.

6.1.2 Baselines. We compare LiveVal against five methods:

- 1) Leave-One-Out (LOO): measures marginal contribution by re-training without each data point. This approach requires retraining the model for each data point, making it computationally expensive.
- 2) Influence Functions (IF) [18]: Approximates LOO through the model's loss change:

$$v_i^{\text{IF}} = -\nabla_{\theta} \ell(f(\mathbf{z}_{\text{test}}; \theta^*), y_{\text{test}})^{\top} H_{\theta^*}^{-1} \nabla_{\theta} \ell(f(\mathbf{x}_i; \theta^*), y_i), \quad (32)$$

where H_{θ^*} is the Hessian matrix at optimal parameters θ^* .

- 3) GradNd [32]: A real-time method that scores training data by expected gradient norm:

$$v_i^{\text{GradNd}} = \mathbb{E}_{\theta_t} [\|\nabla_{\theta} \ell(f(\mathbf{x}_i; \theta_t), y_i)\|_2] \quad (33)$$

- 4) TracIn [33]: A representative gradient-based method using aggregating gradient alignment across training checkpoints $\mathcal{T}$:

$$v_i^{\text{TracIn}} = \sum_{t \in \mathcal{T}} \eta_t \nabla_{\theta} \ell(f(\mathbf{z}_{\text{test}}; \theta_t), y_{\text{test}})^{\top} \nabla_{\theta} \ell(f(\mathbf{x}_i; \theta_t), y_i), \quad (34)$$

- 5) StaticRef: Our basic trajectory method (Section 4.2) measuring optimization guidance toward final parameters θ^* .

Table 1: Comparison of Data Valuation Methods in Terms of Complexity and Characteristics

Method	Time Complexity	Storage Complexity	Real-time	Key Advantages	Main Limitations
LOO	$O(N \cdot T \cdot B \cdot d)$	$O(N \cdot d)$	No	Direct influence measure	Requires N retraining
IF	$O(d^3 + N \cdot d^2)$	$O(d^2)$	No	Efficient LOO approximation	Restrictive assumptions
GradNd	$O(T \cdot B \cdot d)$	$O(d + N)$	Yes	Real-time evaluation	Local gradient focus
TracIn	$O(M \cdot T \cdot d)$	$O(T \cdot d)$	No	Training dynamics capture	Storage intensive
StaticRef	$O(T \cdot B \cdot d)$	$O(T \cdot d)$	No	Trajectory-based insights	Post-hoc analysis
LiveVal	$O(T \cdot B \cdot d)$	$O(\delta_{\max} \cdot d)$	Yes	Real-time valuation, Temporal fairness	Window size selection

Notation: N = dataset size, T = training steps, B = batch size, M = test data, d = parameter dimension, $\delta_{\max}$ = maximum window size.

6.1.3 Evaluation Metrics. Following standard practice, we use corrupted data detection as a proxy task to evaluate: 1) valuation quality through detection accuracy; 2) computational efficiency through wall-clock time; and 3) storage requirements through peak GPU memory consumption via `nvidia-smi`.

All experiments use PyTorch 2.0 on NVIDIA RTX 2080 Ti GPUs with eight random seeds.

6.2 Comparison Against Baselines

6.2.1 Theoretical Comparison. Table 1 compares LiveVal against baseline methods. LOO requires N retrains with $O(N \cdot T \cdot B \cdot d)$ time complexity and $O(N \cdot d)$ storage. IF avoids retraining but relies on expensive $O(d^3)$ Hessian computations. GradNd achieves efficiency but provides only local gradient views. LiveVal enables real-time analysis with storage complexity of only $O(\delta_{\max} \cdot d)$.

6.2.2 Experimental Comparison. This section compares LiveVal against baseline methods in terms of detection accuracy and resource efficiency on flipped label detection tasks.

Experimental Design. We select the MNIST dataset and flip k labels ($k \in \{10, 20, 30, 40\}$) from digit ‘1’ to ‘7’, chosen for their structural similarities, to create a challenging detection scenario. After training CNN on the corrupted dataset, we compute data values on 100 data points (k corrupted data and $(100 - k)$ randomly selected clean data) and check the number of flipped data among the k lowest-valued data. Computational time and graphics memory usage are recorded during the entire valuation process.

Table 2: Comparison of Corruption Detection Performance.

# Flipped	LOO	IF	GradNd	TracIn	StaticRef	LiveVal
10	8.4 ± 2.4	4.6 ± 2.1	0.2 ± 0.4	4.8 ± 2.4	7.4 ± 1.3	5.4 ± 2.6
20	17.4 ± 1.7	11.8 ± 4.4	0.6 ± 0.9	17.5 ± 2.8	14.2 ± 1.1	12.6 ± 4.6
30	27.0 ± 1.4	18.0 ± 4.0	0.8 ± 1.3	26.5 ± 3.3	21.1 ± 3.7	21.3 ± 5.4
40	37.2 ± 2.2	24.2 ± 4.0	1 ± 0.7	34.1 ± 4.5	27.4 ± 1.5	31.2 ± 2.1

Table 3: Comparison of Computational Resource Requirements (Time and GPU Memory).

Metrics	LOO	IF	GradNd	TracIn	StaticRef	LiveVal
Computation time	3h 34m 29s	1m 29s	13m 2s	7m 42s	2m 14s	1m 11s
Graphics memory usage	6.817GB	0.522GB	0.448 GB	1.88 GB	1.24 GB	0.521GB

Results and Analysis. As shown in Table 2 and 3, the experimental results reveal a clear accuracy-efficiency trade-off among different data valuation methods. LOO achieves superior detection accuracy, identifying 8.4 out of 10 corrupted data at the lowest corruption level, but requires prohibitive computational resources with over 3.5 hours of execution time and 6.817 GB memory usage. TracIn demonstrates competitive detection performance but requires substantial computational overhead with 7m 42s execution time and 1.88 GB memory.

LiveVal strikes an effective balance between detection capability and computational efficiency. While it achieves slightly lower detection accuracy than LOO and TracIn, it enables early detection during training and dramatically reduces computational requirements to just 1m11s and 0.521 GB memory usage. This represents a 180× speedup over LOO while maintaining reasonable detection performance. In contrast, influence functions can only detect harmful data after training completion, by this point, malicious data has already corrupted the training process, wasting time and computation resources. GradNd fails at the fundamental detection task, identifying at most 1.0 corrupted data across all corruption levels.

6.3 Early Detection Capability

Theoretical analysis shows that only LiveVal and GradNd support real-time evaluation. Experimental results, however, reveal GradNd’s poor detection performance, leaving LiveVal as the only effective method for early harmful data detection. This section analyzes LiveVal’s early detection performance evolution across training epochs. Figure 3 demonstrates LiveVal’s strong early detection capability. For the $k = 40$ scenario, LiveVal identifies 11 out of 40 corrupted data (27.5%) within the first epoch and reaches 29 data (72.5%) by epoch 3. This early detection capability enables model owners to correct during training before corrupted data significantly impacts model convergence, validating LiveVal’s practical utility for enhancing model robustness.

6.4 Robustness Analysis of Feature Corruption

Beyond label corruption, we evaluate LiveVal’s capability to detect corrupted feature data, which presents more subtle detection challenges than label errors.

Experimental Design. This experiment is specifically designed to assess LiveVal’s sensitivity to different degrees of data corruption and its robustness across varying scales of corruption. Using the MNIST dataset with LeNet-5 model, we inject Gaussian noise

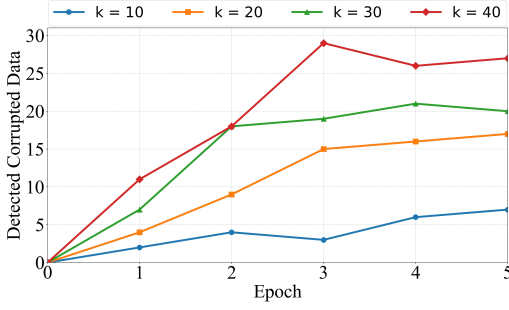


Figure 3: Early Detection Performance of LiveVal across training epochs. The curves show the number of identified corrupted data in the first five epochs for different corruption levels ($k = 10, 20, 30, 40$).

with different standard deviations $\sigma \in \{1.0, 2.0, 5.0\}$ to k randomly selected data ($k \in \{10, 20, 30, 40\}$). We train the LeNet-5 model on this modified dataset and compute data values using LiveVal. We evaluate detection performance by checking how many corrupted data appear in the k lowest-valued data among 100 test data (all k corrupted plus $(100 - k)$ clean data).

Table 4: Number of Feature Corrupted Data Detected under Varying Gaussian Noise Levels ($\sigma \in \{1.0, 2.0, 5.0\}$)

# Corrupted	$\sigma = 1.0$	$\sigma = 2.0$	$\sigma = 5.0$
10	2.0 ± 1.6	2.4 ± 1.1	2.4 ± 1.1
20	6.2 ± 4.4	9.0 ± 2.6	10.2 ± 2.3
30	11.2 ± 6.0	13.2 ± 3.1	18.0 ± 4.2
40	25.8 ± 2.9	28.4 ± 6.2	30.0 ± 4.7

Results and Analysis. Table 4 shows the number of corrupted data successfully detected by LiveVal. First, LiveVal shows strong adaptability to different levels of data corruption, with detection rates increasing as noise magnitude grows (e.g., from 11.2 to 18.0 data at $k = 30$ when σ increases from 1.0 to 5.0). This sensitivity stems from LiveVal’s parameter space analysis: larger noise leads to more significant parameter deviations from the reference trajectory, making corrupted data more distinguishable through their gradient contributions.

Second, LiveVal performs better with larger-scale corruptions, identifying 75% of corrupted data at $k = 40$ compared to 24% at $k = 10$ under $\sigma = 5.0$. This corruption-level dependent behavior arises from LiveVal’s adaptive reference mechanism. When more corrupted data are present, their collective impact creates more distinct parameter trajectories, enabling more reliable detection through reference point comparisons. These properties make LiveVal particularly effective for real-world scenarios where data quality issues often manifest as systematic corruptions affecting multiple data points rather than isolated noise.

6.5 Hyperparameter Sensitivity Analysis

The adaptive window mechanism is at the core of LiveVal’s real-time performance. The thresholds $\epsilon_{\min}$ and $\epsilon_{\max}$ directly control

when the algorithm adjusts its temporal scope. These parameters determine the trade-off between responsiveness to training dynamics and stability against noise.

Experimental Design. Using the same experimental setup as Section 6.2.2, we evaluate LiveVal’s robustness to hyperparameter choices by conducting sensitivity analysis on the adaptive window mechanism parameters $\epsilon_{\min}$ and $\epsilon_{\max}$ under the scenario of 40 flipped labels. We vary $\epsilon_{\min} \in \{0.0001, 0.00032, 0.001, 0.0032, 0.01\}$ and $\epsilon_{\max} \in \{0.01, 0.032, 0.1, 0.32, 1.0\}$ covering three orders of magnitude to capture both conservative and aggressive adaptation behaviors. Other parameters are fixed: learning rate 0.1, $\delta_{\min} = 1$, $\delta_{\max} = 100$, $\Delta\delta = 1$. Each configuration is evaluated using the detection rate metric, which measures the fraction of flipped labels correctly identified.

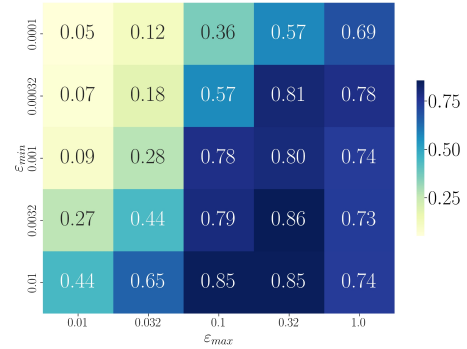


Figure 4: LiveVal Detection Rate Across Different Settings

Results and Analysis. Figure 4 reveals several key insights about the hyperparameters. First, the peak performance (0.86 detection rate) occurs at $\epsilon_{\min} = 0.0032$, $\epsilon_{\max} = 0.32$. This 1:100 ratio suggests that the algorithm has a clear separation between stable and volatile training stages. Second, LiveVal achieves robust performance across reasonable hyperparameter ranges, with detection rates consistently above 0.70 for most middle-range configurations. These results demonstrate the practical robustness of LiveVal. Third, performance degrades predictably at parameter extremes. When $\epsilon_{\min} \leq 0.0001$, the algorithm becomes too conservative, adapting windows too slowly. Detection rates drop to 0.05-0.09 because the method fails to shrink windows during convergence phases, missing fine-grained harmful data patterns. When $\epsilon_{\max} \geq 1.0$, excessive window fluctuations reduce stability, and performance plateaus around 0.70. This is because the method treats normal training fluctuations as volatility, resulting in unnecessary window expansion, which diminishes the reference.

We further track three key metrics throughout training: the training loss L_t , the loss change rate $\dot{L} = (L_t - L_{t-1})/\delta_{t-1}$, and the adaptive window size δ_t . Figure 5 illustrates how LiveVal adjusts δ in response to training dynamics.

Early in training (steps 0–40), with stable loss, the window remains small for high-resolution valuation. During rapid loss descent (steps 40–60), $|\dot{L}|$ exceeds $\epsilon_{\max}$, triggering a larger window for stability. As training converges (steps 60–400), $\dot{L}$ diminishes, and δ adaptively shrinks, maintaining precision with minimal overhead.

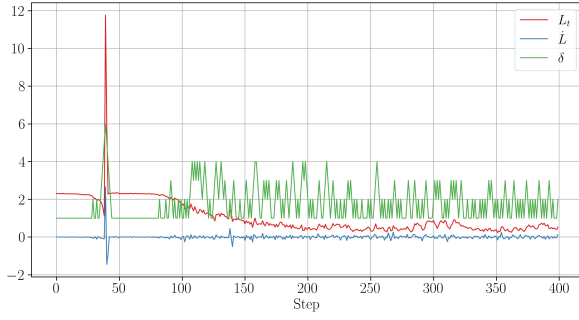


Figure 5: Real-time dynamics of LiveVal’s adaptive window mechanism. The figure shows adaptive responsiveness to different training stages.

This confirms LiveVal’s ability to adjust in real-time without manual tuning.

6.6 Robustness to Different Modalities

Different data modalities present distinct challenges for data valuation due to their inherent structural differences. We evaluate LiveVal’s generalizability across text and tabular data modalities.

Experimental Design. We evaluate LiveVal on two additional datasets: 20 Newsgroups (text) and Adult (tabular), using a DNN architecture. We deliberately flip $k\%$ of labels ($k \in \{10, 20, 30, 40\}$) in each dataset. Following our previous evaluation method, for each k , we evaluate detection performance on 100 data (k flipped plus $(100 - k)$ randomly selected clean data) and examine how many flipped data appear in the k lowest-valued data.

Table 5: Number of Detected Flipped Labels Across Modalities

# Flipped	k=10	k=20	k=30	k=40
20Newsgroups	3.0 ± 1.0	8.8 ± 2.3	16.8 ± 3.4	22.6 ± 2.4
Adult	3.4 ± 2.1	6.8 ± 1.9	11.6 ± 2.4	17.2 ± 3.4

Results and Analysis. Table 5 reveals LiveVal’s varying effectiveness across different data modalities. For text data (20 Newsgroups), LiveVal’s detection rate improves substantially from 30% at $k = 10$ to 56.5% at $k = 40$. This strong scaling behavior can be attributed to the rich feature interactions in text data: corrupted data tend to create more distinct parameter trajectories due to the high-dimensional, sparse nature of text representations. For tabular data (Adult), LiveVal shows more moderate improvement, from 34% at $k = 10$ to 43% at $k = 40$. Despite these modality-specific variations, LiveVal maintains good detection performance across both domains, demonstrating its potential as a general-purpose data valuation framework.

6.7 Scaling to Large Models

Modern deep learning often employs large architectures where traditional data valuation methods become computationally impractical. We evaluate LiveVal on Inception V3 (48 layers with 24M parameters) to show its scalability.

Experimental Design. Following the same label flipping setup from Section 6.2.2, we flip k labels from digit ‘1’ to ‘7’ where $k \in \{10, 20, 30, 40\}$ and train the Inception V3 model using the modified MNIST dataset for one epoch. We evaluate randomly selected 100 data, including k flipped and $(100 - k)$ clean data, and identify the k lowest-valued data.

Table 6: LiveVal’s Detection on Large-scale Model

# Flipped	k=10	k=20	k=30	k=40
# Detected	3.0 ± 2.2	8.6 ± 4.2	11.4 ± 5.9	22.6 ± 2.4
Detect Rate	30%	43%	38%	56%

Results and Analysis. The results demonstrate LiveVal’s effectiveness on large-scale models from multiple aspects. Table 6 shows LiveVal’s detection performance on Inception V3 with 24M parameters. With increasing corruption levels, LiveVal maintains stable detection rates, reaching 56% for $k = 40$ even with just one epoch of training. This early detection capability is crucial for identifying harmful data before they significantly impact the model training.

Beyond detection accuracy, LiveVal also shows significant computational advantages. LiveVal completes the evaluation in 7 min 32s on a single NVIDIA A40 GPU, a remarkable achievement considering Inception V3’s 24M parameters, compared to LOO’s runtime of hundreds of hours. Additionally, LiveVal requires only 31.18GB compared to influence functions’ 2.3TB for Hessian computation. These efficiency gains make LiveVal practical for large-scale architectures where traditional methods become prohibitive.

7 Conclusion

We presented LiveVal, a real-time, trajectory-based data valuation method that overcomes the limitations of traditional post-hoc, loss-based approaches. LiveVal evaluates each data point’s influence by measuring its alignment with adaptive reference points along the optimization trajectory, enabling early detection of harmful data and fair valuation across training stages. The theoretical analysis proves bounded and stable valuations, while experiments demonstrate LiveVal’s ability to identify harmful data early in training, offering significant computational savings for large-scale applications.

Acknowledgments

This work was supported by the Research Grants Council of Hong Kong under Grants CityU C1043-24GF, R6021-20F, R1012-21, C6015-23G, N_CityU139/21, and CRS_HKUST601/24 and by the Innovation and Technology Commission of Hong Kong (ITC) under Mainland-Hong Kong Joint Funding Scheme (MHKJFS) under Grant MHP/135/23. This work was also supported by the InnoHK initiative, the Government of the HKSAR, and the Laboratory for AI-Powered Financial Technologies (AIFT).

Generative AI Usage Disclosure

During manuscript preparation, we used ChatGPT from OpenAI and Claude from Anthropic for polishing. The models provide suggestions for grammatical corrections, phrasing improvements, and restructuring of sentences and paragraphs. No generative AI were used to design the study, analyze data, generate figures and tables, or create original technical content. All suggestions from AI were manually reviewed, edited, and verified by the authors.

References

- [1] Juhan Bae, Nathan Ng, Alston Lo, Marzyeh Ghassemi, and Roger B Grosse. 2022. If influence functions are the answer, then what is the question? *Advances in Neural Information Processing Systems* 35 (2022), 17953–17967.
- [2] S Basu, P Pope, and S Feizi. 2021. Influence Functions in Deep Learning Are Fragile. In *International Conference on Learning Representations (ICLR)*.
- [3] Samyadeep Basu, Xuchen You, and Soheil Feizi. 2020. On second-order group influence functions for black-box predictions. In *International Conference on Machine Learning*. PMLR, 715–724.
- [4] Sang Keun Choe, Hwijee Ahn, Juhan Bae, Kewen Zhao, Minsoo Kang, Youngseog Chung, Adithya Pratapa, Willie Neiswanger, Emma Strubell, Teruko Mitamura, et al. 2024. What is Your Data Worth to GPT? LLM-Scale Data Valuation with Influence Functions. *arXiv preprint arXiv:2405.13954* (2024).
- [5] R Dennis Cook. 1977. Detection of influential observation in linear regression. *Technometrics* 19, 1 (1977), 15–18.
- [6] Ekin D Cubuk, Barret Zoph, Dandelion Mane, Vijay Vasudevan, and Quoc V Le. 2018. Autoaugment: Learning augmentation policies from data. *arXiv preprint arXiv:1805.09501* (2018).
- [7] Dheeru Dua and Casey Graff. 2019. UCI Machine Learning Repository. <http://archive.ics.uci.edu/ml>
- [8] Clement Fung, Chris JM Yoon, and Ivan Beschastnikh. 2018. Mitigating sybils in federated learning poisoning. *arXiv preprint arXiv:1808.04866* (2018).
- [9] Amirata Ghorbani, Michael Kim, and James Zou. 2020. A distributional framework for data valuation. In *International Conference on Machine Learning*. PMLR, 3535–3544.
- [10] Amirata Ghorbani and James Zou. 2019. Data shapley: Equitable valuation of data for machine learning. In *International Conference on Machine Learning*. PMLR, 2242–2251.
- [11] Amirata Ghorbani, James Zou, and Andre Esteva. 2022. Data shapley valuation for efficient batch active learning. In *2022 56th Asilomar Conference on Signals, Systems, and Computers*. IEEE, 1456–1462.
- [12] Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. 2014. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572* (2014).
- [13] Roger Grosse, Juhan Bae, Cem Anil, Nelson Elhage, Alex Tamkin, Amirhossein Tajdini, Benoit Steiner, Dustin Li, Esin Durmus, Ethan Perez, et al. 2023. Studying large language model generalization with influence functions. *arXiv preprint arXiv:2308.03296* (2023).
- [14] Zayd Hammoudeh and Daniel Lowd. 2024. Training data influence analysis and estimation: A survey. *Machine Learning* 113, 5 (2024), 2351–2403.
- [15] Andrew Ilyas, Sung Min Park, Logan Engstrom, Guillaume Leclerc, and Aleksander Madry. 2022. Datamodels: Predicting predictions from training data. *arXiv preprint arXiv:2202.00622* (2022).
- [16] Ruoxi Jia, David Dao, Boxin Wang, Frances Ann Hubis, Nezihe Merve Gurel, Bo Li, Ce Zhang, Costas Spanos, and Dawn Song. 2018. Efficient task specific data valuation for nearest neighbor algorithms. *Proceedings of the VLDB Endowment* 12, 11 (2018), 1610–1623.
- [17] Ruoxi Jia, David Dao, Boxin Wang, Frances Ann Hubis, Nick Hynes, Nezihe Merve Gürel, Bo Li, Ce Zhang, Dawn Song, and Costas J Spanos. 2019. Towards efficient data valuation based on the shapley value. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 1167–1176.
- [18] Pang Wei Koh and Percy Liang. 2017. Understanding black-box predictions via influence functions. In *International Conference on Machine Learning*. PMLR, 1885–1894.
- [19] Pang Wei Koh, Jacob Steinhardt, and Percy Liang. 2022. Stronger data poisoning attacks break data sanitization defenses. *Machine Learning* (2022), 1–47.
- [20] Pang Wei W Koh, Kai-Siang Ang, Hubert Teo, and Percy S Liang. 2019. On the accuracy of influence functions for measuring group effects. *Advances in Neural Information Processing Systems* 32 (2019).
- [21] Yongchan Kwon and James Zou. 2022. Beta Shapley: a Unified and Noise-reduced Data Valuation Framework for Machine Learning. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 8780–8802.
- [22] Yongchan Kwon and James Zou. 2023. Data-OOB: Out-of-bag estimate as a simple and efficient data value. In *International conference on machine learning*. PMLR, 18135–18152.
- [23] Yongchan Kwon and James Y Zou. 2022. Weightedshap: analyzing and improving shapley based feature attributions. *Advances in Neural Information Processing Systems* 35 (2022), 34363–34376.
- [24] Ken Lang. 1995. Newsweeder: Learning to filter netnews. *International Conference on Machine Learning* (1995), 331–339.
- [25] Yann LeCun, Corinna Cortes, Chris Burges, et al. 2010. MNIST handwritten digit database. <http://yann.lecun.com/exdb/mnist>.
- [26] Weida Li and Yaoliang Yu. 2024. One Sample Fits All: Approximating All Probabilistic Values Simultaneously and Efficiently. *arXiv preprint arXiv:2410.23808* (2024).
- [27] Weida Li and Yaoliang Yu. 2024. Robust data valuation with weighted banzhaf values. *Advances in Neural Information Processing Systems* 36 (2024).
- [28] Xuan Luo, Jian Pei, Cheng Xu, Wenjie Zhang, and Jianliang Xu. 2024. Fast Shapley Value Computation in Data Assemblage Tasks as Cooperative Simple Games. *Proceedings of the ACM on Management of Data* 2, 1 (2024), 1–28.
- [29] Yinbin Miao, Ziteng Liu, Hongwei Li, Kim-Kwang Raymond Choo, and Robert H Deng. 2022. Privacy-preserving Byzantine-robust federated learning via blockchain systems. *IEEE Transactions on Information Forensics and Security* 17 (2022), 2848–2861.
- [30] Thien Duc Nguyen, Phillip Rieger, Roberta De Viti, Huili Chen, Björn B Brandenburg, Hossein Yalame, Helen Möllering, Hossein Fereidooni, Samuel Marchal, Markus Miettinen, et al. 2022. {FLAME}: Taming backdoors in federated learning. In *31st USENIX Security Symposium (USENIX Security 22)*. 1415–1432.
- [31] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. 2013. On the difficulty of training recurrent neural networks. In *International conference on machine learning*. Pmlr, 1310–1318.
- [32] Mansheej Paul, Surya Ganguli, and Gintare Karolina Dziugaite. 2021. Deep learning on a data diet: Finding important examples early in training. *Advances in Neural Information Processing Systems* 34 (2021), 20596–20607.
- [33] Garima Pruthi, Frederick Liu, Satyen Kale, and Mukund Sundararajan. 2020. Estimating training data influence by tracing gradient descent. *Advances in Neural Information Processing Systems* 33 (2020), 19920–19930.
- [34] Herbert Robbins and Sutton Monro. 1951. A stochastic approximation method. *The annals of mathematical statistics* (1951), 400–407.
- [35] Andrea Schioppa, Polina Zablotskaia, David Vilar, and Artem Sokolov. 2022. Scaling up influence functions. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36. 8179–8186.
- [36] Lloyd S Shapley. 1953. A value for n-person games. *Contribution to the Theory of Games* 2 (1953).
- [37] Rachael Hwee Ling Sim, Xinyi Xu, and Bryan Kian Hsiang Low. 2022. Data Valuation in Machine Learning: “Ingredients”, Strategies, and Open Challenges. In *International Joint Conference on Artificial Intelligence*. 5607–5614.
- [38] Rachael Hwee Ling Sim, Yehong Zhang, Mun Choon Chan, and Bryan Kian Hsiang Low. 2020. Collaborative machine learning with incentive-aware model rewards. In *International Conference on Machine Learning*. PMLR, 8927–8936.
- [39] Tianshu Song, Yongxin Tong, and Shuyue Wei. 2019. Profit allocation for federated learning. In *2019 IEEE International Conference on Big Data (Big Data)*. IEEE, 2577–2586.
- [40] Qiheng Sun, Xiang Li, Jiayao Zhang, Li Xiong, Weiran Liu, Jinfei Liu, Zhan Qin, and Kui Ren. 2023. Shapleyfl: Robust federated learning based on shapley value. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. 2096–2108.
- [41] Haoru Tan, Sitong Wu, Fei Du, Yukang Chen, Zhibin Wang, Fan Wang, and Xiaojuan Qi. 2023. Data pruning via moving-one-sample-out. In *Advances in Neural Information Processing Systems*. 18251–18262.
- [42] Jiachen T Wang and Ruoxi Jia. 2023. Data banzhaf: A robust data valuation framework for machine learning. In *International Conference on Artificial Intelligence and Statistics*. PMLR, 6388–6421.
- [43] Jiachen T Wang, Prateek Mittal, Dawn Song, and Ruoxi Jia. 2025. Data Shapley in One Training Run. In *International Conference on Learning Representations (ICLR)*.
- [44] Jiachen T Wang, Dawn Song, James Zou, Prateek Mittal, and Ruoxi Jia. 2025. Capturing the Temporal Dependence of Training Data Influence. In *The Thirteenth International Conference on Learning Representations*.
- [45] Tianhao Wang, Johannes Rausch, Ce Zhang, Ruoxi Jia, and Dawn Song. 2020. A principled approach to data valuation for federated learning. *Federated Learning: Privacy and Incentive* (2020), 153–167.
- [46] Mengzhou Xia, Sadhika Malladi, Suchin Gururangan, Sanjeev Arora, and Danqi Chen. 2024. LESS: selecting influential data for targeted instruction tuning. In *Proceedings of the 41st International Conference on Machine Learning*. 54104–54132.
- [47] Xinyi Xu, Lingjuan Lyu, Xingjun Ma, Chenglin Miao, Chuan Sheng Foo, and Bryan Kian Hsiang Low. 2021. Gradient driven rewards to guarantee fairness in collaborative machine learning. *Advances in Neural Information Processing Systems* 34 (2021), 16104–16117.
- [48] Zichun Yu, Spandan Das, and Chenyan Xiong. 2024. Mates: Model-aware data selection for efficient pretraining with data influence models. *Advances in Neural*

- Information Processing Systems* 37 (2024), 108735–108759.
- [49] Chiyuan Zhang, Samy Bengio, Moritz Hardt, Benjamin Recht, and Oriol Vinyals. 2016. Understanding deep learning requires rethinking generalization. *arXiv preprint arXiv:1611.03530* (2016).
- [50] Jiayao Zhang, Qiheng Sun, Jinfei Liu, Li Xiong, Jian Pei, and Kui Ren. 2023. Efficient sampling approaches to shapley value approximation. *Proceedings of the ACM on Management of Data* 1, 1 (2023), 1–24.